FedNH: Tackling Data Heterogeneity in Federated Learning with Class Prototypes

- Improves the local models' performance for both personalization and generalization by combining the uniformity and semantics of class prototypes.
- Tackles data heterogeneity with class imbalance by utilizing uniformity and semantics of class prototypes.
- The proposed FedNH framework addresses data heterogeneity and extreme class imbalance in personalized federated learning by decoupling the neural network into a representation body and a classification head.

1. Prototype Uniformity (Combating Representation Collapse)

To prevent the feature representations of minority classes from being overridden and collapsed into the space of dominant classes due to local class imbalance, an inductive geometric constraint is introduced:

- **Maximally Separated Initialization:** Before federated training begins, the server solves a constrained optimization problem that forces all class prototypes to have a unit norm and maps them uniformly onto a unit hypersphere. This guarantees that all class centers are equidistant and maximally separated from one another.
- **Fixed Local Training Targets:** During the local client update phase, the classification head is frozen. Clients only optimize their local feature extractor bodies to align their data with these predefined, uniform global targets.

2. Semantic Infusion (Preserving Class Relationships)

While strict geometric uniformity protects minority classes, it ignores real-world semantic similarities. To restore these relationships, a non-parametric server update is introduced:

- **Local Representation Aggregation:** After local training, each client computes the average feature embeddings for its available classes and sends them back to the server.
- **Smoothed Global Update:** The server performs a non-parametric update by linearly combining the rigid, uniform geometric structure with the data-driven semantic features aggregated from the clients using a smoothing parameter:

$$W_c^{t+1} \leftarrow \rho W_c^t + (1-\rho) \sum_{k \in S^t} \alpha_k^{t+1} \mu_{k,c}^{t+1}, \qquad (4)$$

FPL: Rethinking Federated Learning with Domain Shift: A Prototype View

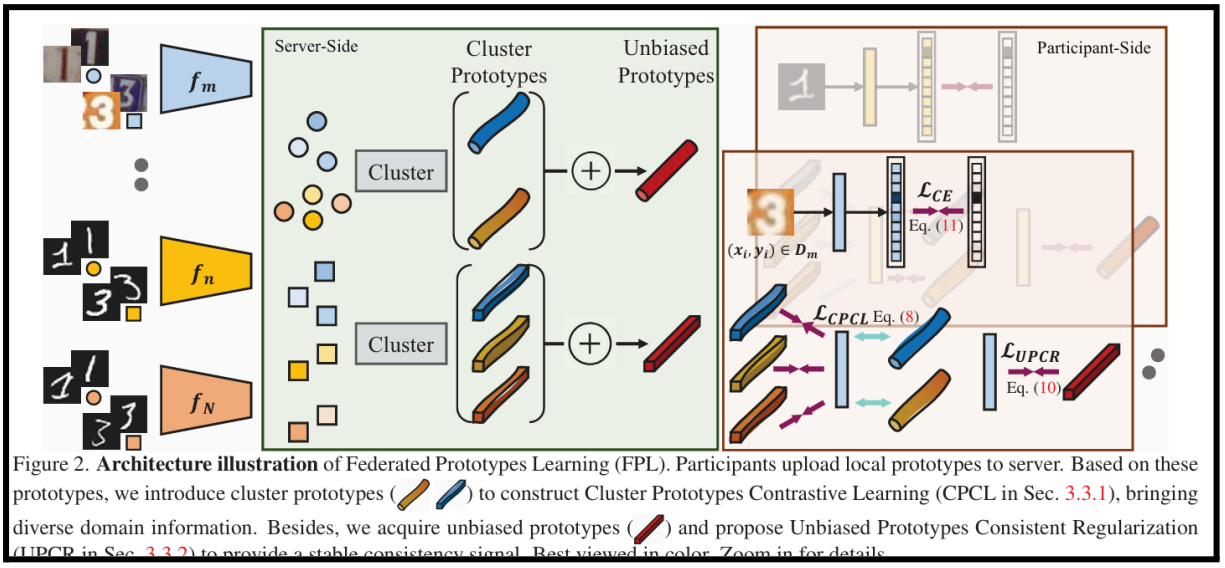

Figure 2. **Architecture illustration** of Federated Prototypes Learning (FPL). Participants upload local prototypes to server. Based on these prototypes, we introduce cluster prototypes ( ) to construct Cluster Prototypes Contrastive Learning (CPCL in Sec. 3.3.1), bringing diverse domain information. Besides, we acquire unbiased prototypes ( ) and propose Unbiased Prototypes Consistent Regularization (UPCR in Sec. 3.3.2) to provide a stable consistency signal. Best viewed in color. Zoom in for details.

- cluster prototypes to provide rich domain knowledge and further construct unbiased prototypes based on the average of cluster prototypes to further offer fair and stable objective signals.

FPL resolves domain shift problems by introducing a dual-prototype strategy at the server level, combining Cluster Prototypes and Unbiased Prototypes to ensure both generalization and training stability.

1. Cluster Prototypes via CPCL

Instead of condensing an entire class into a single global representation, the server preserves domain diversity by extracting multiple representative targets:

- **Parameter-Free Clustering:** The server collects local prototypes from all clients and applies the FINCH algorithm. FINCH groups the N participant prototypes of class k into J representative cluster prototypes (P^k) based on first-neighbor cosine similarity.
- **Cluster Prototypes Contrastive Learning (CPCL)**

2. Unbiased Prototypes via UPCR

Because unsupervised clustering is dynamic, the number and structure of cluster prototypes change across communication epochs, which can cause training instability. FPL introduces an optimization anchor to stabilize training:

Calculating Unbiased Targets: For each class, the server takes the arithmetic mean of the generated cluster prototypes, rather than the raw client updates

- By averaging the clusters directly, dominant/primary domains with massive client representation are prevented from skewing the target direction, producing an unbiased objective.

- Unbiased Prototypes Consistent Regularization: Local models minimize the feature-level Mean Squared Error distance between the query instance and its corresponding unbiased prototype. This establishes a stable convergence target across training rounds.

FedPLVM: Taming Cross-Domain Representation Variance in Federated Prototype Learning with Heterogeneous Data Domains

- Existing FedPL methods create the same number of prototypes for each client, leading to **cross-domain performance gaps** and disparities for clients with varied data distributions.

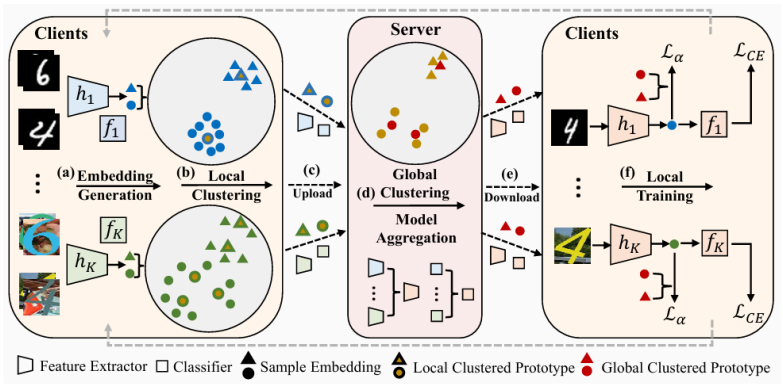

Figure 2: **An overview of our proposed FedPLVM framework. Once** the sample embedding is generated by the feature extractor, the client conducts the first-level local clustering, following Eq. 3. **Subsequently**, the server gathers all local clustered prototypes and local models (comprising feature extractors and classifiers), initiates the second-level global clustering based on Eq. 4, and averages the local models to form a global model. **Finally**, clients utilize the received global clustered prototypes to update the local model, employing loss functions $\mathcal{L}_\alpha$ from Eq. 5 and $\mathcal{L}_{CE}$ from Eq. 9.

Contrastive Learning. Contrastive learning is a promising self-supervised learning technique. The work by [25] constructs pairs of positives and negatives for each sample and applies the InfoNCE loss to compare these pairs. Another work by [15] extends contrastive learning from self-supervised to fully supervised settings, utilizing both label information and contrastive methods. Additionally, some studies [16, 39] integrate contrastive learning into local training to enhance performance in FL. Instead of the conventional InfoNCE loss, our approach introduces a new α-sparsity loss, aiming to further reduce similarity among inter-class sample features while amplifying similarity among intra-class samples.

- A dual-level prototypes clustering strategy creates local clustered prototypes based on private data features, then performs global prototypes clustering to reduce communication complexity and preserve local data privacy.

FedPLVM Methodology Overview

FedPLVM is designed to tackle heterogeneous data domains and disparate learning difficulties (the performance gap between "easy" and "hard" domains) in Federated Prototype Learning . Instead of wiping out data diversity by averaging feature representations into single vectors, it preserves domain variance across the entire distributed system.

1. Dual-Level Prototype Generation

Rather than creating an identical number of prototypes or a single mean vector for every client, FedPLVM uses a two-stage clustering pipeline to capture true distribution widths:

- **First-Level: Local Prototype Clustering (Client-Side):** Once local training concludes, each client passes its samples through its local feature extractor. Instead of taking a simple arithmetic mean of all features within a class, the client applies the parameter-free FINCH clustering algorithm independently for each category.
- **Second-Level: Global Prototype Clustering (Server-Side):** Sending an unchecked, massive assortment of local prototypes from every client to the network would clog communication bandwidth and leak client-specific private attributes . To prevent this, the server aggregates all incoming local prototypes and runs a second round of clustering per class . This downscales the overall prototype variety into a refined, compact set of Global Clustered Prototypes, boosting both efficiency and data privacy .

2. α-Sparsity Prototype Loss

- **The Contrastive Term:** It applies a fractional power factor to the positive cosine similarity scores . This concave transformation creates exceptionally steep gradients for small similarity values, forcing the network to heavily penalize cross-class overlaps and aggressively repel competing classes to establish clean, sparse geometric boundaries .
- **The Corrective Term:** Because this transformation unintentionally stretches intra-class distances, this component acts as a counterweight . It measures the cumulative similarity between an instance and its true global prototypes, pulling them back toward maximum alignment to keep same-class features tightly bound .

FedTGP: Trainable Global Prototypes with Adaptive-Margin-Enhanced Contrastive Learning for Data and Model Heterogeneity in Federated Learning

Optimization Strategy (Adaptive-Margin Contrastive Learning)

- **Contrastive Objective:** The server optimizes the global prototypes by pushing identical classes together (**Semantic Alignment**) and forcing different classes apart (**Separation**).
- **Dynamic Gap-Sizing (Adaptive Margin):** Enforcing a rigid, large separation distance early on degrades training since client feature extractors are initially weak. To solve this, the server measures the maximum natural distance achieved between client classes in the current round and uses that exact distance as the target separation margin.

End-to-End Execution Loop (Per Round)

1. **Broadcast:** The server selects a subset of clients and sends them the current refined global prototypes.
2. **Local Client Training:** Clients train their local models using their own private data. They minimize standard classification loss while applying a regularization penalty if their local features drift too far from the server's global prototypes.
3. **Classwise Aggregation:** After training, each client calculates a single local prototype for each class by taking the classwise mean of all feature vectors belonging to that specific class.
4. **Upload & Server Update:** Clients upload only these lightweight class averages back to the server (protecting raw data and model IP). The server updates its dynamic margin and runs its processing network to refine the global prototypes for the next round.

FedPall: Prototype-based Adversarial and Collaborative Learning for Federated Learning with Feature Drift

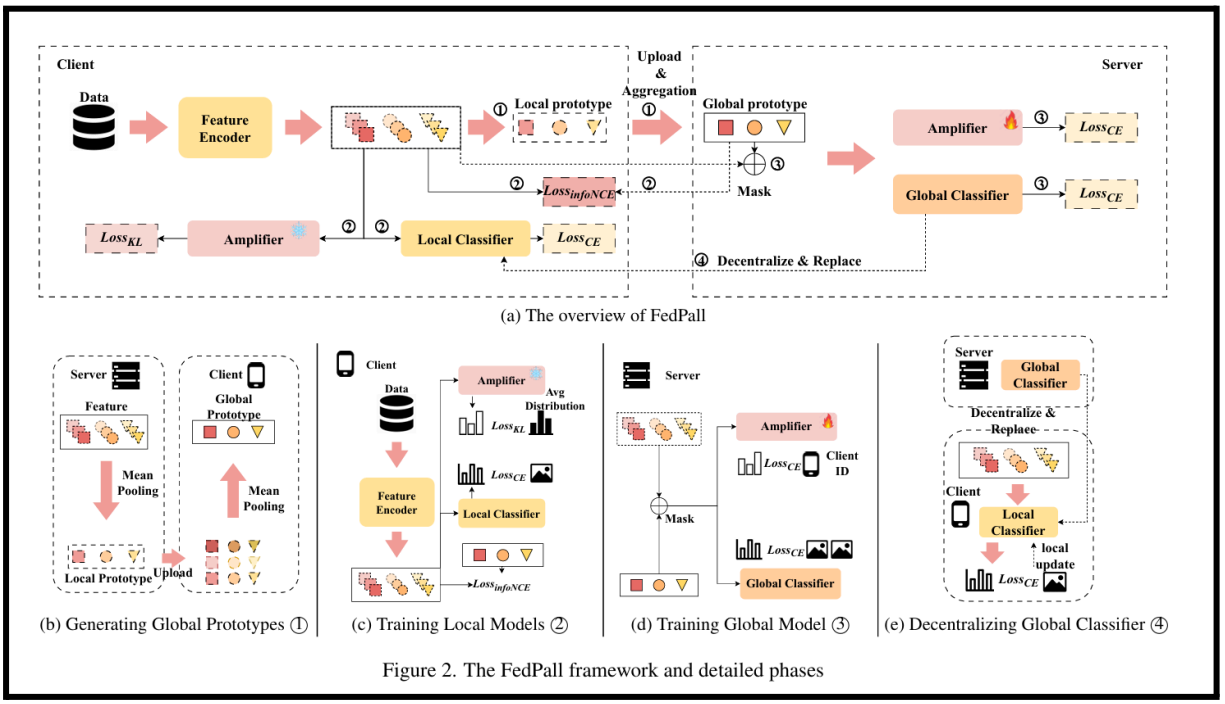

Figure 2. The FedPall framework and detailed phases

The Core Problem: Feature Drift

Feature drift happens when different clients collect data for the same exact classes, but the underlying data looks completely different due to varying devices, environments, or lighting (e.g., MNIST digits vs. SVHN street numbers vs. blurry synthetic digits).

1. Generating Global Prototypes

- **Local Classwise Means:** Each client processes its private data through its local feature extractor and computes the mean of the feature vectors for each class to generate local prototypes.

- **Server Aggregation:** Clients upload these local prototypes alongside their local class sizes. The server performs a size-weighted average to create the global prototypes and broadcasts them back.

2. Collaborative Learning (Reinforcing Class Geometries)

- **Contrastive Guidance:** Clients utilize the server's global prototypes as reference anchors during local training.
- **InfoNCE Alignment:** Using the InfoNCE loss, local feature representations are pulled closer to their corresponding global prototype while being pushed further away from the prototypes of all other incorrect classes. This step protects class discriminative details from blurring.

3. Adversarial Learning (Eliminating Feature Drift)

- **The Problem:** Contrastive alignment alone fails to erase domain/device-specific style artifacts (feature drift).
- **The Server's Role (Policing):** The server trains a global Amplifier network to look at feature vectors and predict their respective client ID (identifying device-specific artifacts).
- **The Client's Role (Scrubbing):** The clients use KL Divergence loss against a frozen copy of this Amplifier. Clients optimize their feature extractors to systematically scrub domain-specific markers, confusing the Amplifier until features from all clients are aligned into a unified distribution.

4. Prototype Mixing & Global Classifier (Secure Global Decision Boundaries)

- **The Bottleneck:** Clients have limited local data and cannot build optimal standalone classifier layers on their own.
- **Prototype Mixing:** To prevent privacy leaks from uploading raw features, clients linearly blend their local features with the global prototypes via a random ratio.
- **Bernoulli Masking:** A random Bernoulli Mask is applied to the blended features, dropping certain dimensions to zero to render the embeddings irreversible and safe for transmission.
- **Global Server Training:** The server aggregates these masked, hybrid features to train a robust Global Classifier with a broad, cross-client perspective.
- **Decentralization & Personalization:** The server distributes this Global Classifier to replace the local classifiers. Clients run a final short retraining step (fine-tuning) on their clean local data to personalize the model to their local distribution.

FedDAP: Domain-Aware Prototype Learning for Federated Learning under Domain Shift

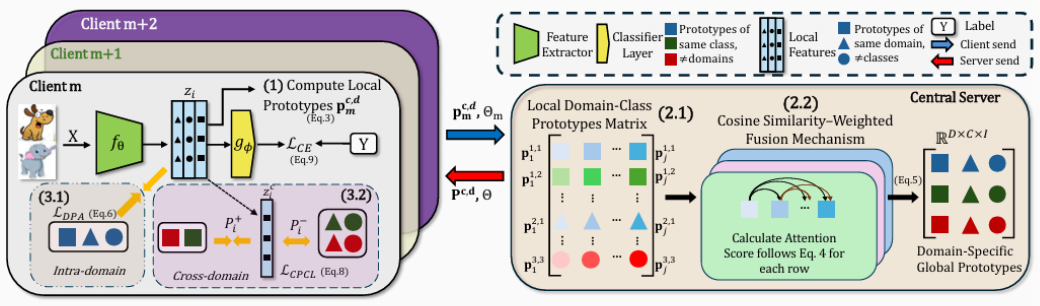

Figure 2. **Illustration of FedDAP.** (1) Clients first compute local prototypes $\mathbf{p}_m^{(c,d)}$ based on Eq. 3 and then upload it to server. (2) The central server performs the cosine similarity-weighted fusion mechanism for the local prototypes by calculating the attention score in Eq. 4 for each row of class-domain pair prototypes. Then we perform the Domain-Specific Global Prototype aggregation using Eq. 5. (3) Clients download the global prototypes $\mathbf{P}^{c,d}$ from the server and perform the local training process with the dual prototype alignment strategy with $\mathcal{L}_{DPA}$ in Eq. 6 and $\mathcal{L}_{CPCL}$ in Eq. 8.

1. Constructing Domain-Specific Global Prototypes

- **Preserving Domain Semantics:** FedDAP maintains a distinct global prototype for every unique (class, domain) pair.
- **Local Feature Computation:** Each individual client calculates its standard local prototypes by finding the classwise mean of the feature vectors generated by its local encoder.
- **Uploading with Identifiers:** Clients pack these local prototypes and explicitly upload them to the central server alongside a Domain Identifier (such as "Photo", "Cartoon", or "Sketch").

2. Cosine Similarity-Weighted Fusion (Server Aggregation)

- **Selective Grouping:** The server organizes all incoming client prototypes according to their strict class-domain matches.
- **Attention-Based Weighting:** Instead of standard uniform averaging, the server computes an attention score using the cosine similarity between client prototypes within the same group.
- **Filtering Anomalies:** Higher optimization weights are assigned to client entries that exhibit strong semantic consistency with their style peers. This structurally protects the aggregated prototypes from being skewed by outliers or poorly trained local networks.

3. Dual Prototype Alignment Strategy (Local Client Training)

Once the organized domain-specific global tensor is distributed, clients utilize a tailored Dual Alignment Strategy to guide their local feature extractors along two distinct axes:

- **Domain-Consistent Prototype Alignment (Intra-Domain):**
  - The local model forces its generated features to align strictly with the global prototype that matches both its correct target class and its own domain style.
  - cosine similarity

- **Cross-Domain Prototype Contrastive Learning (Cross-Domain):**
    - Simultaneously, a contrastive objective forces local features to be pulled toward matching class representations belonging to other domains.
    - Concurrently, it aggressively pushes those features away from prototypes representing completely different classes across other domains.

FedProIn: Mitigating Client Drift for Learnable Prototypes in Federated Medical Imaging

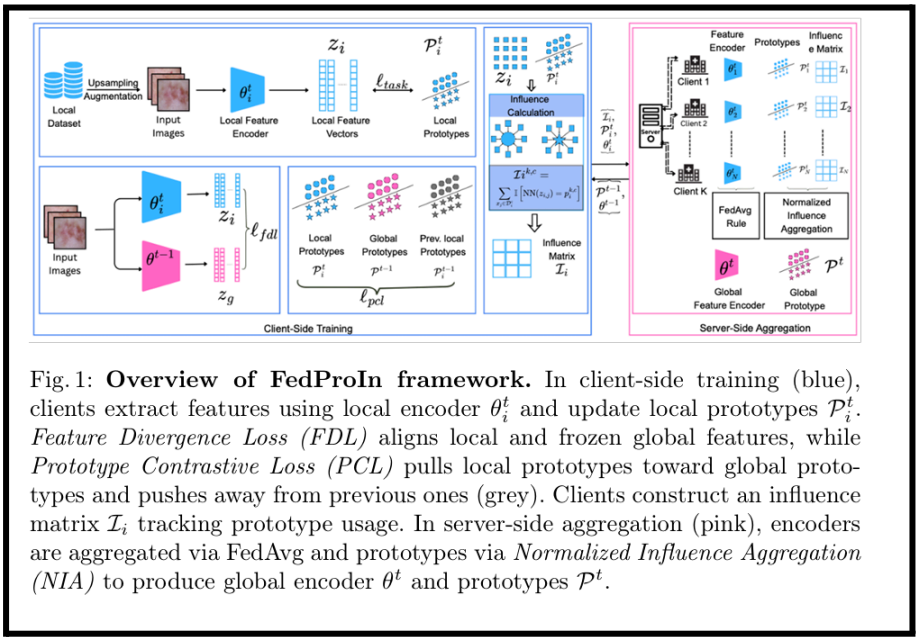

Fig. 1: **Overview of FedProIn framework.** In client-side training (blue), clients extract features using local encoder θ_i^t and update local prototypes $\mathcal{P}_i^t$. *Feature Divergence Loss (FDL)* aligns local and frozen global features, while *Prototype Contrastive Loss (PCL)* pulls local prototypes toward global prototypes and pushes away from previous ones (grey). Clients construct an influence matrix $\mathcal{I}_i$ tracking prototype usage. In server-side aggregation (pink), encoders are aggregated via FedAvg and prototypes via *Normalized Influence Aggregation (NIA)* to produce global encoder θ^t and prototypes $\mathcal{P}^t$.

- **Feature drift:** Client models map images to different regions in feature space due to data heterogeneity.
- **Prototype drift**: Client prototypes are pulled towards local data characteristics, causing prototypes to diverge from global representation.

Core Problem: Client Drift in Heterogeneous Environments

- **Statistical Heterogeneity:** In medical federated learning, different clients use different scanners, acquisition protocols, and face varying patient populations. This means the data distributed across the network is highly non-IID.
- **Client Drift:** When clients train a shared global model locally on their skewed data, their model parameters diverge in completely different optimization directions. When the central server averages these models, it leads to unstable convergence and suboptimal performance.

The framework breaks this massive "client drift" problem down into two structural failure modes:

1. **Feature Drift:** Heterogeneous data causes different client models to map identical clinical concepts/diseases to completely distinct, misaligned regions in the high-dimensional embedding space.

2. **Prototype Drift:** Client-side class prototypes get pulled too close to local data biases and anomalies, causing them to deviate from a generalized global representation.
3. **Imbalance Stagnation:** In long-tailed medical datasets, standard contrastive gradients only update the absolute closest prototypes. Prototypes representing rare disease subtypes receive zero gradients, causing them to freeze and fail to learn their semantic regions.

Global Objective

The framework divides the global model into two separate, interacting pieces:

    - **Feature Encoder:** A deep neural network that takes raw medical images as input and transforms them into low-dimensional feature vectors within an embedding space.
    - **Learnable Prototypes:** Rather than calculating static, temporary average vectors from data samples after training, this method creates a fixed number of dedicated, trainable class prototypes that act directly as model parameters optimized via backpropagation.
- **Global Optimization Goal:** The system aims to minimize the overall empirical risk by aggregating updates across all decentralized client datasets.

Client-Side Local Training & Dual Regularization

To explicitly mitigate the overall client drift caused by variations in medical imaging environments (like different scanners or patient populations), the framework breaks the problem down into two distinct sub-components and treats them with separate regularization steps:

A) Feature Divergence Loss

- **Target:** Combats feature drift, a phenomenon where different client models map identical medical conditions into entirely different areas of the embedding space due to data distribution shifts.
- **Mechanism:** At the start of a training round, each client gets a copy of the current global feature encoder, which is frozen and kept unchanged during local training.
- **Logic:** The client's active local model is forced to align its generated feature representations closely with the reference features produced by the frozen global model. This ensures that the way the local model interprets images does not wildly drift away from the global consensus.

B) Prototype Contrastive Loss

- **Target:** Combats prototype drift, where local class prototypes get pulled too close to client-specific data biases, causing them to lose their general clinical meaning.
- **Mechanism & Logic:** The framework uses a contrastive strategy that simultaneously pulls the active local prototypes closer to the generalized global prototypes from the previous round. At the same time, it actively pushes them away from the client's own old local states from the prior round.

- **Margin Control:** A margin hyperparameter is used to enforce a strict boundary, ensuring that the updated local prototypes remain closer to the global template than to their own historical local baselines.

C) Data Imbalance Handling

- Clients leverage class-balanced sampling and adaptive data augmentation. By oversampling minority classes and creating varied visual views, they force the network to actively select and update these rare subregion prototypes.

Server-Side Aggregation Protocol

When a local training round concludes, the server collects the updated encoder parameters, updated prototypes, and an influence tracking matrix from each participating client. The server aggregates these components using two completely different strategies:

A) Feature Encoder Aggregation

- The image-processing encoder layers are combined using the standard FedAvg protocol.
- The server takes a standard weighted average of the client encoder weights based strictly on the respective dataset sizes of the participating centers.

B) Normalized Influence Aggregation for Prototypes

- **Why Traditional Aggregation Fails:** Denzeling multi-prototype setups with standard FedAvg or mean averaging ruins performance for two reasons:
    1. Heterogeneous Prototype Utilization: Within a single disease class, distinct prototypes are not activated at the same frequency. Averaging them purely based on total client dataset size amplifies background noise from underutilized or poorly trained prototypes.
    2. Sparse Prototype Activation: Centers that lack specific rare disease subtypes will completely leave those corresponding prototypes unutilized. Standard averaging would mistakenly grant these unoptimized prototypes full weight, drowning out the highly specialized insights trained by the few clients who actually possess those rare samples.
- **The NIA Mechanism:** During local training, clients maintain a running tally (an influence matrix) counting exactly how many times each specific class prototype served as the absolute closest match for a local training sample.
- **Global Synthesis:** Instead of looking at total dataset sizes, the server uses these real-time utilization counts as weights. Prototypes are averaged proportionally to their actual usage during training. This automatically dampens noisy updates from underutilized components while preserving highly specialized prototypes across heterogeneous clinical networks.

FedAdapterProto: Prototype-Guided and Lightweight Adapters for Inherent Interpretation and Generalisation in Federated Learning

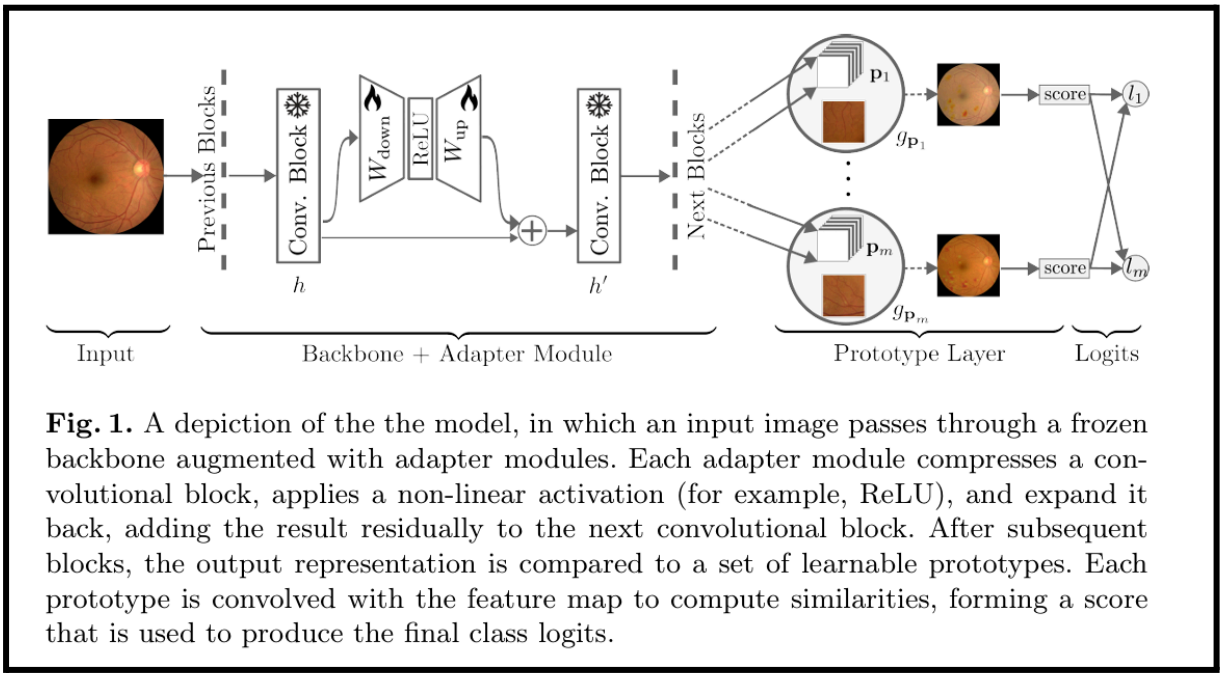

Fig. 1. A depiction of the the model, in which an input image passes through a frozen backbone augmented with adapter modules. Each adapter module compresses a convolutional block, applies a non-linear activation (for example, ReLU), and expand it back, adding the result residually to the next convolutional block. After subsequent blocks, the output representation is compared to a set of learnable prototypes. Each prototype is convolved with the feature map to compute similarities, forming a score that is used to produce the final class logits.

1. Core Framework Strategy & Philosophy

The framework is designed to handle the challenges of **statistical heterogeneity (varying clinical site data)** and **high communication overhead**.

- **The Invariant Core:** The heavy, primary feature-extracting backbone of the model is kept completely frozen locally across all clients.
- **The Commuting Subsets:** Optimization and server synchronization are strictly restricted to low-dimensional local adapters and part-based interpretability prototypes.

2. Local Client Architecture

Instead of a standard unified neural network, each client runs a hybrid architecture containing three distinct layers:

A. Frozen Backbone

- Utilizes a standard deep convolutional backbone architecture (ResNet-50).
- All weights in this backbone are entirely locked during local optimization steps.

B. Lightweight Adapter Modules

- These are highly compact bottleneck networks inserted at the terminal end of every primary convolutional block within the frozen backbone.

- **Squeeze and Expand Mechanism:** An incoming feature map from the backbone is first compressed into a much lower dimensional space to strip out unnecessary data and prevent local overfitting. It passes through a non-linear activation and is then expanded back to its original shape.
- **Residual Connection:** The adjusted features are added directly back to the original incoming features. This skip-connection bypasses the main block to prevent vanishing gradients and helps the model adapt to unique local image qualities (like lighting or camera variations) without altering the main brain.

C. Prototype Layer

- This replaces standard fully-connected linear classification layers with an interpretable spatial template matcher.
- It contains a set of learnable, class-specific reference prototypes that represent characteristic visual parts of different classes from the training data.
- **Similarity Matching:** The layer scans the final adjusted feature map coming out of the adapted backbone. It measures the geometric distance between regions of the input image and these ideal disease templates.
- **Heatmap Generation:** Because this matching happens region by region, the layer outputs an activation map. This map can be upsampled into a visual heatmap (showing red boxes or glowing regions) that highlights exactly which parts of a medical image (such as lesion markers) triggered the classification decision.

3. Local Training Objectives

During local training epochs, the frozen backbone stays locked. The client updates its local adapters and prototypes using a multi-part compound loss function designed to enforce balance:

1. **Classification Accuracy:** Forces the model to predict the correct clinical label using standard cross-entropy.
2. **Adapter Regularization:** Constrains the weights of the small adapter modules to prevent them from over-adjusting and overfitting to the local site.
3. **Prototype Clustering:** Pushes representations of an image closer to its correct-class prototype template while actively driving them away from wrong-class prototypes.
4. **Global Proximal Alignment:** Acts as an anchor by penalizing the local adapters and prototypes if they drift too far away from the global reference parameters shared by the server. This ensures the client maintains a connection to the collective "common knowledge".

FedHPro: Federated Hyper-Prototype Learning via Gradient Matching

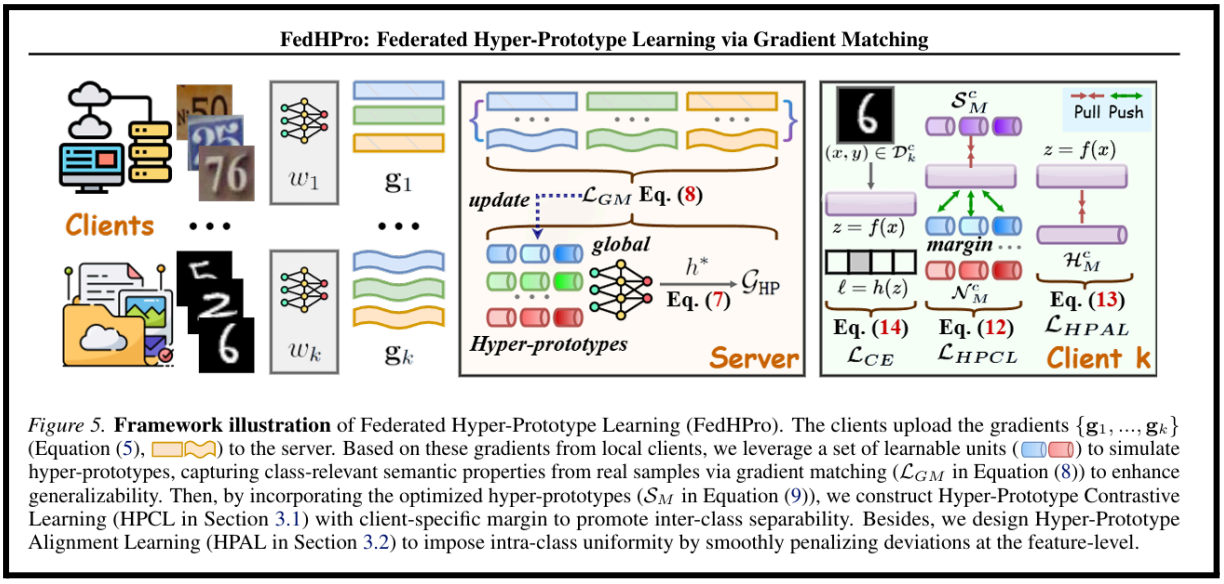

Figure 5. **Framework illustration** of Federated Hyper-Prototype Learning (FedHPro). The clients upload the gradients $\{\mathbf{g}_1, ..., \mathbf{g}_k\}$ (Equation (5),) to the server. Based on these gradients from local clients, we leverage a set of learnable units ( ) to simulate hyper-prototypes, capturing class-relevant semantic properties from real samples via gradient matching ($\mathcal{L}_{GM}$ in Equation (8)) to enhance generalizability. Then, by incorporating the optimized hyper-prototypes ($\mathcal{S}_M$ in Equation (9)), we construct Hyper-Prototype Contrastive Learning (HPCL in Section 3.1) with client-specific margin to promote inter-class separability. Besides, we design Hyper-Prototype Alignment Learning (HPAL in Section 3.2) to impose intra-class uniformity by smoothly penalizing deviations at the feature-level.

1. The Core Innovation: Hyper-Prototypes

- Instead of averaging biased local data, the central server initializes its own set of learnable, unbiased global anchors called **Hyper-Prototypes**.

2. Server-Side Methodology: Gradient Matching

- **How the Server Learns (Without Seeing Data):** To train these Hyper-Prototypes without violating privacy, clients process their real data and calculate "gradients" (which act as directional instructions for how the model should update). Clients upload these gradients to the server instead of prototypes.
- **The Matching Process:** The server uses **a technique called Gradient Matching. It mathematically adjusts its Hyper**-Prototypes so that their optimization path perfectly mimics the exact gradients sent by the clients.
- **The Result:** The server successfully distills the true semantic characteristics of the data directly from real samples, creating a highly accurate global guide as if it had trained on all the data centrally.

3. Client-Side Methodology: Local Training

Once the server creates the perfect Hyper-Prototypes, it sends them to the clients to guide their local training. The clients use two specific modules:

A. Hyper-Prototype Contrastive Learning (HPCL) - Focuses on Separation

- **Goal:** To promote **inter-class separability** (ensuring the AI doesn't confuse two different categories).
- **Mechanism:** HPCL uses contrastive learning with an adaptive, "client-specific margin" to sharpen the decision boundaries. It forces the local model to pull a sample's features closer to its correct hyper-prototype and strongly push it away from the hyper-prototypes of all other incorrect categories.

B. Hyper-Prototype Alignment Learning (HPAL) - Focuses on Consistency

- **Goal:** To promote **intra-class uniformity** (ensuring the same category looks identical across different, heterogeneous clients).
- **Mechanism:** Even after separating classes, different clients might still interpret the same category slightly differently due to their unique data. HPAL applies a smooth regularization penalty that strictly forces every client's local features to perfectly align with the exact center of the global hyper-prototype. This provides a stable, uniform target for everyone

1. Broadcasting (Server → Clients) To start the round, the central server randomly selects a subset of participating clients. The server sends these clients the current global model parameters and the globally optimized **Hyper-Prototypes**. (Note: In the very first round, the model and hyper-prototypes are initialized by the server).
2. Local Updating & Gradient Calculation (Client-Side) The clients receive the global model and hyper-prototypes and use them to train on their private, local data over a set number of epochs. During this local training step, the client optimizes its model using a combined loss function that includes three parts:

- **Cross-Entropy Loss:** Standard training for classification.
- **Hyper-Prototype Contrastive Learning (HPCL):** The client uses the received hyper-prototypes to pull its data samples closer to the correct category's hyper-prototype and push them away from incorrect ones, creating sharp decision boundaries.
- **Hyper-Prototype Alignment Learning (HPAL):** The client applies a penalty to strictly align its feature representations with the exact center of the global hyper-prototypes, ensuring its data perfectly matches the global standard.

Once local training epochs are complete, the client computes the average gradients for each class directly from its real data samples.

3. Uploading (Clients → Server) Instead of sending raw data or biased local prototypes, each participating client uploads two things to the central server:

- Its updated **local model parameters**.
- The **class-wise average gradients** it just calculated.

4. Global Aggregation & Gradient Matching (Server-Side) Once the server receives the uploads from all participating clients, it performs two critical aggregation tasks:

- **Model Aggregation:** The server averages all the local model parameters together to generate the new, updated global model.
- **Hyper-Prototype Optimization:** The server first averages all the class-wise gradients submitted by the clients to figure out the true semantic direction of the data. It then uses **Gradient Matching** to optimize its Hyper-Prototypes. It mathematically adjusts the Hyper-Prototypes so that their optimization path explicitly mimics the averaged gradients from the real client samples.

5. Round Completion The server now possesses a newly updated global model and a freshly optimized set of Hyper-Prototypes that accurately reflect the real data without having compromised privacy. These are held ready to be broadcast to the clients to initiate the next communication round.

ProtoNorm: Heterogeneous Federated Learning with Prototype Alignment and Upscaling

- Designed to handle heterogeneous model architectures and non-IID (non-independent and identically distributed) data across clients

1. Local Prototype Generation (Client Side): For each category (class) of data it holds, the client calculates the mean activation vector in the penultimate layer. This average feature vector is called the local prototype for that class, which is then transmitted to the central server.

2. Simple Averaging (Server Side):

Traditional PBFL methods use a weighted average based on the number of samples each client holds, which skews the global prototypes toward clients with the most data. To overcome this and prevent privacy leaks regarding a client's specific data distribution, ProtoNorm uses a simple average of all the received local prototypes to create the initial global prototypes.

3. Phase 1: Prototype Alignment / PA (Server Side):

Once the initial global prototypes are averaged, the server initiates the first core phase of the framework to maximize their angular separation.

- **Hyperspherical Projection:** The global prototypes are mathematically normalized so that they are placed on the surface of a unit hypersphere.
- **Thomson Problem Optimization:** Inspired by classical physics, the system treats these prototypes like electrons that naturally repel one another to find

the lowest energy state on a sphere. The server minimizes the "hyperspherical energy" of these points.

- **Gradient Ascent:** Using a gradient ascent-based algorithm, the server calculates the repulsive "force" between all pairs of prototypes. It iteratively updates their positions using velocity and momentum until they reach a minimal energy configuration, resulting in perfect angular separation on the sphere.

4. Phase 2: Prototype Upscaling / PU (Server to Client Transition):

While the PA phase ensures optimal angular spacing, global prototypes bounded to a unit sphere (magnitude of 1) are often too small. If used directly, they can cause the clients' local training to converge to suboptimal local minima because they do not match the natural activation scale of the models' feature extractors.

- To solve this, the methodology applies a scaling hyperparameter (γ).
- By multiplying the perfectly aligned prototypes by γ, the system artificially upscales their magnitudes, pulling them outward and vastly enhancing their separation in Euclidean space.

5. Local Model Regularization (Client Side):

The server sends these optimally aligned and upscaled global prototypes back to the clients. When clients train their local models on their private data, they do not just use standard classification loss. Instead, they use a combined loss function.

- This combined loss includes a specific regularization term that calculates the Euclidean distance between the client's local features and the upscaled global prototypes.
- By minimizing this distance, the local models are forcefully guided to align their own learned representations with the perfectly separated global targets, thereby overcoming local data skewness and significantly improving overall classification accuracy